Analyze and select, based on scientific and technical literature published in recent years, ten (10) years, machine learning and computer vision models oriented to the detection of diseases and pests in rice crops, prioritizing criteria of computational efficiency, operation in offline mode and compatibility with entry-level mobile devices. a. Deliverable: i. Benchmarking executive report, including comparison of Models, advantages, limitations and technical justification of the proposals selected.

For rice disease detection on entry-level mobile devices prioritizing computational efficiency and offline operation, YOLO-tiny variants (particularly YOLOv4 tiny and Improved YOLOv5s) targeting 3-6 disease classes represent the optimal selection, achieving over 95% detection accuracy with sub-second inference times and proven offline deployment capability, while Vision Transformers and large CNN architectures exceed resource constraints of entry-level devices despite higher accuracy potential.

Abstract

YOLO-based architectures demonstrate optimal performance for rice disease detection on entry-level mobile devices when prioritizing computational efficiency and offline operation. YOLOv4 tiny achieves 97.36% mAP with approximately 5 ms inference time and full offline capability through TensorFlow Lite conversion, while Improved YOLOv5s delivers 96.1% mAP with 47.5% parameter reduction and 374 ms per frame processing on Android devices with 4GB RAM. Lightweight CNN architectures provide alternative solutions, with Simple CNN achieving 93.3% accuracy while reducing model size by 99% compared to VGG16. Detection performance exhibits systematic trade-offs: models targeting 3-6 disease classes consistently achieve >95% mAP, while comprehensive 13-class systems reach only 69% mAP, reflecting increased inter-class similarity challenges rather than model limitations. Mobile deployment viability requires models under 15MB size, sub-350ms inference time, and under 300MB RAM usage for entry-level devices, achievable through optimization frameworks including NCNN, TensorFlow Lite, and MNN acceleration. Vision Transformers and DenseNet variants achieve superior accuracy (99.38% and 98.03% respectively) but face computational constraints limiting deployment to edge computing devices with GPU acceleration rather than entry-level mobile platforms. For offline field operation on resource-constrained devices, YOLO-tiny variants targeting focused disease sets (3-6 classes) represent the optimal selection based on demonstrated accuracy-efficiency balance and proven mobile deployment.

Paper search

We performed a semantic search across over 138 million academic papers from the Elicit search engine, which includes all of Semantic Scholar and OpenAlex.

We ran this query: ”Analyze and select, based on scientific and technical literature published in recent years, ten (10) years, machine learning and computer vision models oriented to the detection of diseases and pests in rice crops, prioritizing criteria of computational efficiency, operation in offline mode and compatibility with entry-level mobile devices. a. Deliverable: i. Benchmarking executive report, including comparison of Models, advantages, limitations and technical justification of the proposals selected.”

The search returned 50 total results from Elicit.

We retrieved 50 papers most relevant to the query for screening.

Screening

We screened in sources based on their abstracts that met these criteria:

- **AI-Based Approach**: Does the study focus on machine learning and/or computer vision models for automated detection (rather than traditional non-AI image processing methods)?
- **Rice Crop Application**: Is the target application disease and/or pest detection in rice crops (either exclusively or with rice-specific validation)?
- **Performance Metrics and Technical Details**: Does the study provide both quantitative performance evaluation (accuracy, precision, recall, F1-score, etc.) AND sufficient technical details about the model architecture?
- **Mobile/Resource Efficiency**: Is the model designed for or tested on mobile/edge devices OR does it demonstrate computational efficiency metrics suitable for resource-constrained environments (e.g., low processing time, memory usage, model size, FLOPs)?
- **Real-world Validation**: Does the study include validation on real rice crop datasets with practical implementation or testing (not just theoretical frameworks)?
- **Offline Operation Capability**: Does the research include offline operation capability or potential for deployment without requiring constant internet connectivity?
- **Field-based Detection Focus**: Does the study focus on field-based crop monitoring rather than being limited solely to post-harvest disease detection?
- **Methodological Detail Adequacy**: Is this a full research paper with sufficient methodological detail for technical analysis (not just a conference abstract, poster, or paper with insufficient detail)?

We considered all screening questions together and made a holistic judgement about whether to screen in each paper.

Data extraction

We asked a large language model to extract each data column below from each paper. We gave the model the extraction instructions shown below for each column.

- **Model Architecture**:

  Extract details about the machine learning/computer vision models used for rice disease and pest detection, including:

- Model name and type (e.g., YOLOv5s, MobileNet, CNN architecture)
- Base architecture or framework used
- Specific modifications or improvements made to the base model
- Novel architectural components (e.g., attention mechanisms, lightweight modules)
- Model variants compared in the study

- **Computational Efficiency**:

  Extract all computational efficiency metrics for rice disease/pest detection models, including:

- Model size (MB, parameters count)
- Floating Point Operations (FLOPs)
- Inference speed (ms/frame, FPS)
- Memory usage/requirements

- Processing time per image
- Computational complexity comparisons with baseline models
- Percentage improvements in efficiency metrics

- **Mobile Implementation**:

  Extract information about mobile device compatibility and deployment for rice disease/pest detection, including:

- Specific mobile devices/platforms tested (Android, iOS, Raspberry Pi)
- Hardware specifications of test devices
- Mobile application development details
- Real-time performance on mobile devices
- Optimization techniques used for mobile deployment (quantization, pruning, acceleration engines)
- Entry-level device compatibility mentioned

- **Detection Performance**:

  Extract accuracy and detection performance metrics for rice disease and pest identification, including:

- Accuracy, precision, recall, F1-score
- Mean Average Precision (mAP) at different IoU thresholds
- Detection success rates
- Performance comparisons with other models
- Performance variations by disease/pest type
- Validation dataset results

- **Offline Capability**:

  Extract information about offline operation capability for rice disease/pest detection models, including:

- Whether the model operates without internet connection
- Local processing capabilities
- Edge computing implementation
- Data storage requirements for offline operation
- Any cloud dependency mentioned
- Real-time field operation capability

- **Disease Coverage**:

  Extract the scope of rice diseases and pests that each model can detect, including:

- Specific diseases detected (e.g., brown spot, leaf blast, bacterial blight)
- Number of disease/pest classes
- Pest types identified (e.g., hispa, insect damage)
- Disease severity levels detected
- Generalization to other crops mentioned

- **Dataset Details**:

  Extract information about datasets used for training and testing rice disease/pest detection models, including:

- Dataset size (number of images)
- Image collection methods and sources

- Data augmentation techniques used
- Training/validation/test split ratios
- Image quality and resolution specifications
- Real-world vs. controlled environment data

- **Advantages Limitations**:

  Extract stated advantages and limitations of each rice disease/pest detection model approach, including:

- Key strengths and benefits highlighted by authors
- Technical limitations acknowledged
- Computational constraints
- Environmental or operational limitations
- Comparison advantages over existing methods
- Scalability issues mentioned
- Practical deployment challenges

Results

Characteristics of Included Studies

The ten sources examined represent primary research studies and systematic reviews published between 2018 and 2025, focusing on machine learning models for rice disease and pest detection with varying emphases on mobile deployment.

Study	Full text retrieved?	Study Type	Model Architecture	Mobile Platform	Dataset Size	Disease/Pest Classes
Jizhong Deng et al., 2023	Yes	Primary study	Improved YOLOv5s, Improved YOLOv7-tiny	Android	3,300 images	6 classes
C. R. Rahman et al., 2018	Yes	Primary study	VGG16, InceptionV3, Simple CNN, MobileNet, NasNet Mobile, SqueezeNet	Not specified	1,426 images	9 classes
Bimarsha Khanal et al., 2024	Yes	Primary study	YOLOv8, Vision Transformer, CNN models (ResNet, VGG, MobileNet)	ReactNative (cross-platform)	16,225 images	13 classes

Study	Full text retrieved?	Study Type	Model Architecture	Mobile Platform	Dataset Size	Disease/Pest Classes
A. Nayak et al., 2023	No	Primary study	DenseNet201, Xception, MobileNetV2, ResNet50	Android	2,259 training images, 250 validation images	12 classes
Mahrin Tasfe et al., 2025	No	Review	DenseNet, InceptionV3, Xception, MobileNet, ViT, parallel-ViT, ensemble models; segmentation models	Not specified	Novel dataset created	Not specified
Wei Ye et al., 2025	No	Primary study	TG_YOLOv5	Raspberry Pi	Self-constructed dataset	Not specified
Md. Mehedi Hasan et al., 2023	Yes	Primary study	Novel CNN architecture	Android	2,700 training, 1,200 validation, 630 test images	3 classes
Siddhi Jain et al., 2022	Yes	Primary study	YOLOv3 tiny, YOLOv4 tiny	Android	762 images	3 classes
T. P et al., 2023	No	Primary study	Dense CNN models, MobileNet models, ResNet family	Mobile devices	Collected from field and repositories	6 classes
Apri Junaidi et al., 2025	No	Systematic review	YOLOv7, Faster R-CNN, YOLOv8, Vision Transformers	Edge devices (NVIDIA Jetson Nano)	Review of 76 articles	Not specified

The studies span diverse methodological approaches, with seven primary research papers and three systematic reviews. Most studies targeted Android platforms for mobile deployment, while others explored Raspberry Pi and edge computing devices. Dataset sizes varied considerably, from 762 images to over 16,000 images, with disease coverage ranging from 3 classes to 13 classes. Five studies had full text available for detailed extraction, while five were analyzed based on abstracts only.

Model Architectures and Technical Specifications

Model Family	Specific Variants	Novel Components	Computational Improvements	Parameter Reductions
YOLO-based	Improved YOLOv5s, Improved YOLOv7-tiny, TG_YOLOv5, YOLOv3 tiny, YOLOv4 tiny, YOLOv8	Ghost module, CBAM and SIoU, Triplet attention and C3CBAM, GhostConv module	Model size reduced by 45.7%, FLOPs reduced by 48.7%	Parameters reduced by 47.5%
CNN Architectures	VGG16, InceptionV3, Simple CNN, Novel CNN	Two-stage training	Simple CNN 99% smaller than VGG16	Simple CNN has 0.8M parameters vs 138M in VGG16
Transfer Learning	DenseNet201, Xception, MobileNetV2, ResNet50	Not specified	MobileNetV2 optimized for smartphones	Not specified
Lightweight	MobileNet, NasNet Mobile, SqueezeNet	Memory-efficient design	Dense CNN and MobileNet require fewer resources than ResNet	Not specified
Transformers	Vision Transformer, parallel-ViT	Patch embedding layer	High computational demands noted	Not specified

The YOLO-based architectures demonstrated substantial computational optimizations through architectural innovations. The Improved YOLOv5s achieved parameter reduction of 47.5%, model size reduction of 45.7%, and FLOPs reduction of 48.7% compared to the original YOLOv5s, while simultaneously increasing inference speed by 38.6%. These improvements were primarily attributed to the Ghost module integration, which reduces computation while maintaining detection accuracy. Similarly, TG_YOLOv5 incorporated the GhostConv lightweight module to reduce model complexity, combined with triplet attention mechanisms and C3CBAM modules to enhance feature extraction without significantly increasing parameters.

Among traditional CNN architectures, the Simple CNN demonstrated exceptional size efficiency, achieving 99% size reduction compared to VGG16 with only 0.8 million parameters versus VGG16's 138 million. This dramatic reduction was accomplished through a two-stage training approach, making it particularly suitable for resource-constrained mobile devices.

Detection Performance and Accuracy

Model	Precision	Recall	F1-Score	mAP (0.5)	mAP (0.5:0.95)	Inference Speed	Disease Accuracy by Type
Improved YOLOv5s	Improved by 3.2%	Improved by 0.3%	0.931	0.961	0.648	374 ms/frame	Better for large targets
Improved YOLOv7-tiny	Not specified	Not specified	Not specified	Second to YOLOv5s	Not specified	Slower than YOLOv5s	Better for small targets
TG_YOLOv5	Not specified	97.2%	Not specified	98.3%	Not specified	73.8% faster with MNN	Not specified
Simple CNN	Not specified	Not specified	Not specified	Not specified	Not specified	Not specified	93.3% overall
Fine-tuned VGG16	Not specified	Not specified	Not specified	Not specified	Not specified	Not specified	97.12% overall
YOLOv8	Not specified	Not specified	Not specified	69%	Not specified	Not specified	Multiple diseases detected
Vision Transformer	Not specified	Not specified	Not specified	Not specified	Not specified	Not specified	99.38% classification
DenseNet201	Not specified	Not specified	Not specified	Not specified	Not specified	Not specified	98.03% validation
MobileNetV2	Not specified	Not specified	Not specified	Not specified	Not specified	Not specified	97.56% validation
Novel CNN	Not specified	Highest for leaf smut	Highest for brown spot	Not specified	Not specified	103 ms	97.9% overall
YOLOv4 tiny	0.88	0.89	0.89	97.36%	Not specified	~5 ms/image	Brown spot: 97.20%
YOLOv3 tiny	0.77	0.79	0.74	79.77%	Not specified	~3 ms/image	Leaf blast: 99.48%
YOLOv7	Not specified	Not specified	Not specified	99%	Not specified	0.0549 s on Jetson Nano	Not specified

Detection performance metrics revealed considerable variation across model architectures and deployment contexts. Among YOLO-based models, YOLOv7 achieved the highest reported mAP at 99%, followed by TG_YOLOv5 at 98.3% and YOLOv4 tiny at 97.36%. The Improved YOLOv5s demonstrated balanced performance with an F1-score of 0.931 and mAP of 0.961 at IoU threshold 0.5, with particularly strong detection on large target areas. In contrast, the Improved YOLOv7-tiny showed superior accuracy for small target detection, suggesting complementary strengths between these architectures.

Classification models achieved exceptionally high accuracies, with Vision Transformer reaching 99.38%, though at the cost of computational intensity. Among transfer learning approaches, DenseNet201 achieved 98.03% validation accuracy, followed by Xception at 97.78% and MobileNetV2 at 97.56%. Disease-specific performance varied, with the

Novel CNN achieving perfect 100% accuracy for leaf smut detection and 97.2% for bacterial leaf blight. YOLOv4 tiny demonstrated consistent detection across disease types, with leaf blast at 99.48%, hispa at 98.45%, and brown spot at 97.20%.

The significant performance advantage of YOLOv4 tiny over YOLOv3 tiny (97.36% vs 79.77% mAP) represents a 17.59% improvement, highlighting the impact of architectural evolution within the same model family. However, YOLOv8's lower mAP of 69% suggests that detection tasks involving multiple simultaneous disease identification present distinct challenges compared to single-disease classification scenarios.

Mobile Deployment and Real-Time Performance

Platform	Model	Hardware Specifications	Processing Speed	Memory Usage	Offline Capability	Optimization Techniques
Android	Improved YOLOv5s	Android 9.0, 4GB RAM	374 ms/frame	CPU: 49%, RAM: 262.9MB	Yes	NCNN framework
Android	MobileNetV2	Not specified	Not specified	Not specified	Yes	Transfer learning
Android	Novel CNN	Not specified	103 ms	Minimal resources	Yes	TensorFlow Keras
Android	YOLOv4 tiny	Not specified	~5 ms/image	Not specified	Yes	TensorFlow Lite
ReactNative	YOLOv8 + ViT	Not specified	Not specified	Not specified	Cloud-dependent	Distributed architecture
Raspberry Pi	TG_YOLOv5	Not specified	73.8% faster with MNN	Not specified	Yes	MNN engine acceleration
Edge device	YOLOv7-tiny	NVIDIA Jetson Nano	0.0549 s/image	Not specified	Yes	Edge computing

Mobile deployment strategies varied significantly across studies, with most focusing on Android platforms. The Improved YOLOv5s demonstrated practical viability on Android devices with moderate specifications (Android 9.0, 4GB RAM), achieving real-time processing at 374 ms per frame with reasonable resource consumption (49% CPU usage, 262.9MB RAM). The NCNN framework optimization proved critical for enabling efficient on-device inference.

Edge computing implementations showed promising results, with YOLOv7-tiny on NVIDIA Jetson Nano achieving 0.0549 seconds latency per image, demonstrating the feasibility of real-time detection on specialized edge hardware. The Raspberry Pi deployment of TG_YOLOv5 achieved a 73.8% speed increase through MNN engine acceleration, validating the approach for low-cost edge computing scenarios.

Offline capability varied across implementations. The Improved YOLOv5s, MobileNetV2, Novel CNN, and YOLOv4 tiny all demonstrated full offline operation through local processing, eliminating internet dependency. TensorFlow Lite conversion enabled on-device processing without network connectivity. However, some implementations maintained cloud dependencies, with the YOLOv8-based system using RabbitMQ for server communication, suggesting partial reliance on network connectivity.

Disease and Pest Coverage

Study	Total Classes	Specific Diseases	Pest Types	Detection Scope
Jizhong Deng et al., 2023	6	Rice smut, rice blast, streak disease, sheath blight	Cnaphalocrocis medinalis, Chilo suppressalis	High-frequency diseases and pests
C. R. Rahman et al., 2018	9	Bacterial leaf blight, brown spot, sheath blight, sheath rot, neck blast, false smut	Brown plant hopper, hispa, stemborer	Comprehensive disease and pest coverage
Bimarsha Khanal et al., 2024	13	Brown spot, blast, downy mildew, bacterial leaf streak, bacterial leaf blight, bacterial panicle blight, tungro	Yellow stem borer, black stem borer, leaf roller, white stem borer, hispa	Fungal, bacterial, viral diseases and major pests
A. Nayak et al., 2023	12	Not specified	Not specified	Diseases and nutrient deficiency symptoms
Md. Mehedi Hasan et al., 2023	3	Rice brown spot, rice bacterial blight, leaf smut	Not included	Limited to three major diseases
Siddhi Jain et al., 2022	3	Brown spot, leaf blast	Hispa	Focused on three common conditions
T. P et al., 2023	6	Blast, bacterial leaf blight, brown spot, false smut, tungro	Not specified	Five diseases plus healthy classification

Disease coverage ranged from focused three-class systems to comprehensive 13-class implementations. The most frequently detected diseases across studies included brown spot, bacterial leaf blight, blast/leaf blast, and tungro. Among pests, hispa appeared consistently, along with various stem borers.

The Bimarsha Khanal et al. study provided the most comprehensive coverage with 13 distinct classes, encompassing fungal diseases (brown spot, blast, downy mildew), bacterial diseases (bacterial leaf streak, bacterial leaf blight, bacterial panicle blight), viral disease (tungro), and five pest types. This breadth enabled multiple simultaneous disease detection, though at the cost of lower mAP (69%) compared to more focused implementations.

Studies with narrower disease scopes achieved higher accuracy rates. The Novel CNN targeting only three diseases (rice brown spot, bacterial blight, leaf smut) achieved 97.9% overall accuracy, with perfect 100% detection for leaf smut. Similarly, YOLOv4 tiny focusing on three conditions achieved 97.36% mAP, suggesting that constraining the detection scope enhances model performance within the targeted disease set.

One study included nutrient deficiency symptoms alongside disease classification, representing an expanded diagnostic scope beyond infectious conditions. However, specific diseases or deficiencies were not detailed in the available abstract.

Synthesis

The heterogeneity in model performance and deployment approaches across studies reflects distinct design philosophies optimized for different operational contexts rather than contradictory findings. Three primary factors explain the observed variations: (1) the accuracy-efficiency trade-off in model architecture selection, (2) the breadth versus depth trade-off in disease coverage, and (3) hardware platform constraints on achievable performance.

Architecture-Performance Trade-offs by Deployment Context

The apparent contradiction between high-performing transformer models (Vision Transformer at 99.38% accuracy) and simpler CNN architectures (Simple CNN at 93.3%) resolves when considering deployment contexts. Vision Transformers achieve superior classification accuracy but face significant computational and data efficiency challenges, making them impractical for entry-level mobile devices. In contrast, lightweight architectures sacrifice marginal accuracy for substantial computational efficiency gains—Simple CNN achieves 99% size reduction compared to VGG16 while maintaining 93.3% accuracy, representing a viable trade-off for resource-constrained scenarios.

The YOLO family demonstrates consistent optimization for mobile deployment across model generations. YOLOv4 tiny outperforms YOLOv3 tiny by 17.59% mAP while maintaining ultra-fast inference (~5 ms per image), validating the architectural evolution within this family. However, YOLOv8's lower detection mAP of 69% in multi-disease scenarios suggests that the task complexity—simultaneous detection of 13 disease classes—rather than model capability drives the observed performance. Studies targeting 3-6 diseases consistently achieve >95% mAP, supporting the hypothesis that detection complexity rather than model inadequacy explains YOLOv8's lower metrics.

Disease Coverage Scope Effects on Model Performance

The inverse relationship between disease coverage breadth and detection accuracy reflects fundamental machine learning principles rather than model limitations. Comprehensive systems detecting 13 disease classes achieve 69% mAP, while focused 3-class systems reach 97.36% or higher. This pattern holds across architectures: the Novel CNN targeting three diseases achieves 97.9% accuracy with 100% accuracy on leaf smut, while broader systems show more variable performance. The explanation lies in inter-class similarity—rice diseases often present visually similar symptoms (lesions, discoloration, structural changes), increasing classification difficulty as the number of classes grows. Studies explicitly acknowledging detection challenges with "small and overlapping lesion detection" support this interpretation.

The advantage of detection models over classification models in multi-disease scenarios reflects their architectural design: detection models process multiple objects simultaneously within single images, while classification models require separate processing for each suspected condition. This architectural difference, not model superiority, explains why "detection models are effective at identifying multiple diseases simultaneously with less computing power" compared to classification approaches.

Hardware Platform Constraints on Achievable Performance

The divergent performance metrics between edge devices (NVIDIA Jetson Nano achieving 0.0549 seconds per image) and entry-level Android phones (374 ms per frame) reflect hardware capability differences rather than optimization failures. Edge computing devices with dedicated GPU acceleration naturally achieve faster inference than consumer mobile devices relying on general-purpose CPU/GPU combinations. The 73.8% speed increase achieved through MNN engine acceleration on Raspberry Pi demonstrates that software optimization can partially bridge this gap, but fundamental hardware differences remain.

The computational efficiency improvements in Improved YOLOv5s—reducing parameters by 47.5%, model size by 45.7%, and FLOPs by 48.7%—represent genuine advancement enabling mobile deployment. These metrics establish a

benchmark: models requiring <15MB size, <350ms inference time, and <300MB RAM demonstrate practical viability on entry-level Android devices with 4GB RAM. Models exceeding these thresholds require either edge computing hardware or sacrifice real-time performance.

Offline Operation Requirements and Architecture Selection

The consistent implementation of offline capability in YOLO-based and lightweight CNN models, contrasted with cloud-dependent Vision Transformer implementations, reflects not architectural necessity but deployment priority decisions. Models converted to TensorFlow Lite or optimized with NCNN framework operate fully offline with local processing, while distributed architectures using RabbitMQ maintain cloud connectivity for model updates and distributed inference. Both approaches are valid: offline operation suits field conditions with limited connectivity, while cloud-connected systems enable continuous model improvement through dynamic framework updates.

The selection of MobileNetV2 over ResNet50 for smartphone deployment, despite ResNet50's suitability for cloud architectures, exemplifies architecture-context matching. MobileNetV2's 97.56% validation accuracy suffices for practical deployment while enabling offline operation, whereas ResNet family models achieve higher accuracy but require "increased computational resources" incompatible with mobile constraints.

Synthesis Conclusions

The evidence supports three operational contexts with distinct optimal model selections:

1. **Entry-level mobile devices (≤4GB RAM, offline operation)**: YOLO-tiny variants (YOLOv4 tiny achieving 97.36% mAP, ~5ms inference) or lightweight CNNs (Simple CNN at 93.3% accuracy, 99% size reduction) provide the best balance of accuracy, efficiency, and offline capability. Disease coverage should be limited to 3-6 classes for >95% detection accuracy.
2. **Mid-range mobile devices (≥4GB RAM, offline operation)**: Optimized YOLO models (Improved YOLOv5s at 96.1% mAP, 374ms inference) or MobileNetV2 (97.56% accuracy) enable higher accuracy with moderate computational demands. Disease coverage can extend to 6-12 classes with acceptable accuracy trade-offs.
3. **Edge computing devices (cloud connectivity acceptable)**: YOLOv7 (99% mAP), Vision Transformers (99.38% accuracy), or DenseNet variants (98.03% accuracy) maximize detection accuracy for comprehensive disease monitoring (10+ classes) with real-time performance on specialized hardware.

References

A. Nayak, S. Chakraborty, and D. Swain. "Application of Smartphone-Image Processing and Transfer Learning for Rice Disease and Nutrient Deficiency Detection." *Smart Agricultural Technology*, 2023.

Apri Junaidi, Siti Zaiton Mohd Hashim, Mohd Shahizan Bin Othman, Mohd Murtadha Mohamad, Hitham Alhussian, S. J. Abdulkadir, Maged Nasser, and Yunusa Adamu Bena. "Deep Learning and Edge Computing in Agriculture: A Comprehensive Review of Recent Trends and Innovations." *IEEE Access*, 2025.

Bimarsha Khanal, Paras Poudel, Anish Chapagai, Bijan Regmi, Sitaram Pokhrel, and S. Khanal. "Paddy Disease Detection and Classification Using Computer Vision Techniques: A Mobile Application to Detect Paddy Disease." *arXiv.org*, 2024.

C. R. Rahman, P. S. Arko, Mohammed Eunus Ali, Mohammad A. Iqbal Khan, S. H. Apon, Farzana Nowrin, and Abu Wasif. "Identification and Recognition of Rice Diseases and Pests Using Convolutional Neural Networks." *Biosystems Engineering*, 2018.

Jizhong Deng, Chang Yang, Kanghua Huang, Luocheng Lei, Jiahang Ye, Wen Zeng, Jianling Zhang, Yubin Lan, and Yali Zhang. "Deep-Learning-Based Rice Disease and Insect Pest Detection on a Mobile Phone." *Agronomy*, 2023.

Mahrin Tasfe, Akm Nivrito, Fadi Al Machot, Mohib Ullah, Faouzi Alaya Cheikh, and Habib Ullah. "Deep Learning-Based Models for Paddy Disease Classification and Segmentation: An Experimental Review." *IEEE Access*, 2025.

Md. Mehedi Hasan, Touficur Rahman, A. Uddin, S. Galib, Mostafijur Rahman Akhond, Md. Jashim Uddin, and Md. Alam Hossain. "Enhancing Rice Crop Management: Disease Classification Using Convolutional Neural Networks and Mobile Application Integration." *Agriculture*, 2023.

Siddhi Jain, Rahul Sahni, Tuneer Khargonkar, Himanshu Gupta, Om Prakash Verma, T. Sharma, Tushar Bhardwaj, Saurabh Agarwal, and Hyunsung Kim. "Automatic Rice Disease Detection and Assistance Framework Using Deep Learning and a Chatbot." *Electronics*, 2022.

T. P, and B. Baranidharan. "Resource Prudent CNN Models for Disease Identification of Rice Crops." *2023 International Conference on Networking and Communications (ICNWC)*, 2023.

Wei Ye, Fei Jiang, Zhaoxing Li, Lei Zhao, Jiaoyu Wang, and Hongkai Wang. "A Study on Real-Time Detection of Rice Diseases in Farmlands Based on Multi-Dimensional Data Fusion." *Plant Disease*, 2025.